Sécurité Web avec Node.js & Express

Module R401 — Sécurité des Applications Web

BUT Informatique — Semestre 4

IUT Nord Franche-Comté — Belfort-Montbéliard

Enseignant : **Joseph Azar**

joseph.azar@umlp.fr

Plan du cours

Ce que nous allons couvrir aujourd'hui

1. **Partie 1** : Protection au niveau infrastructure — API Gateway, NGINX, Reverse Proxy
2. **Partie 2** : Protection au niveau applicatif — Helmet.js, Rate Limiting, CORS, HPP
3. **Partie 3** : OWASP Top 10 — Comprendre les 10 vulnérabilités les plus critiques
4. **Partie 4** : Exemples pratiques avec Node.js & Express — Code vulnérable vs. code sécurisé
5. **Partie 5** : TP Capture The Flag — Mettez vos compétences à l'épreuve !

Pourquoi la sécurité web est cruciale

Chaque application web est une cible potentielle

- En 2025, **plus de 40 000 vulnérabilités** ont été publiées (CVE)
- Les applications Node.js sont particulièrement exposées : npm compte plus de 2 millions de packages, dont beaucoup contiennent des failles
- Une seule vulnérabilité peut compromettre les données de milliers d'utilisateurs
- La sécurité n'est pas une option, c'est une **responsabilité** de chaque développeur

Architecture de défense en profondeur

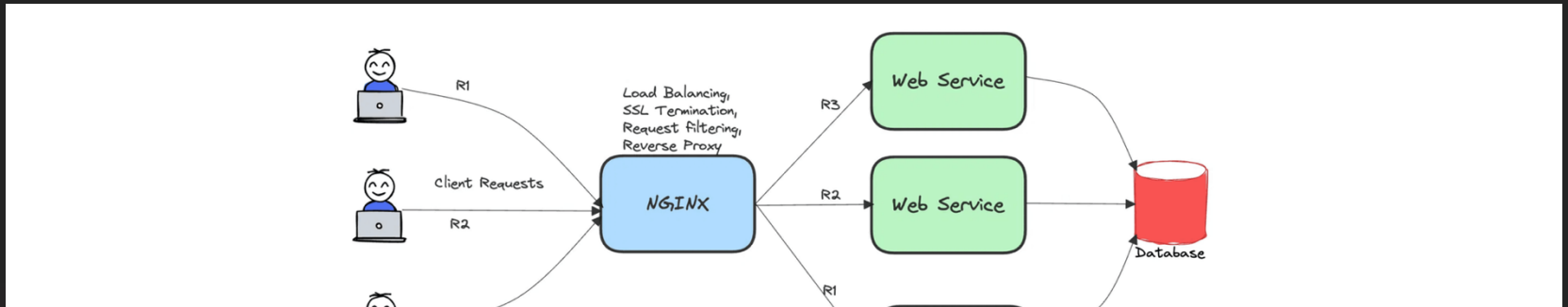
La sécurité se construit en couches successives

- Concept de **Defense in Depth** : plusieurs niveaux de protection
- Couche 1 : **Réseau** — Firewall, DNS, CDN
- Couche 2 : **Infrastructure** — NGINX, API Gateway, Load Balancer
- Couche 3 : **Application** — Helmet, Rate Limiting, Validation des entrées
- Couche 4 : **Données** — Chiffrement, Hashing, Contrôle d'accès
- Diagramme montrant les 4 couches empilées avec le flux d'une requête HTTP traversant chaque couche

Qu'est-ce qu'un Reverse Proxy ?

NGINX comme bouclier devant votre application

- Un **reverse proxy** est un serveur intermédiaire entre le client et le serveur backend
- Le client ne communique jamais directement avec Node.js
- NGINX reçoit les requêtes, les filtre, et les transmet au backend
- Diagramme séquentiel : Client → NGINX (port 80/443) → Node.js (port 3000)
- Avantages : sécurité renforcée, performances améliorées, terminaison SSL



Configuration NGINX comme Reverse Proxy

Mise en place pas à pas

```
server {
    listen 80;
    server_name monsite.fr;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

- Fichier de configuration NGINX :

Rate Limiting avec NGINX

Limiter le nombre de requêtes pour bloquer les attaques

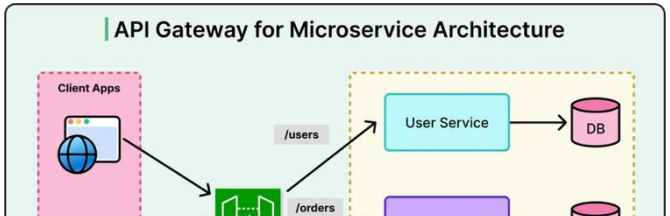
```
http {
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

    server {
        location /api/ {
            limit_req zone=api burst=20 nodelay;
            proxy_pass http://localhost:3000;
        }
    }
}
```

- Le **rate limiting** empêche un utilisateur d'envoyer trop de requêtes en peu de temps
- Protection contre : attaques DDoS, brute force, abus d'API
- Configuration NGINX :

API Gateway : le gardien de vos microservices

Centraliser la sécurité de vos APIs

- Un **API Gateway** est un point d'entrée unique pour toutes les requêtes API
- Fonctions : authentification, rate limiting, logging, routage, transformation des requêtes
- Diagramme : Client → API Gateway → Service Auth / Service Users / Service Products
- Solutions populaires : Kong, Express Gateway, AWS API Gateway, Traefik
- L'API Gateway applique les politiques de sécurité **avant** que la requête n'atteigne le service



Helmet.js : sécuriser les en-têtes HTTP en une ligne

Les en-têtes HTTP comme première ligne de défense

```
const express = require('express');
const helmet = require('helmet');
const app = express();

app.use(helmet()); // Active 12 protections par défaut !
```

- **Helmet.js** est un middleware Express qui configure automatiquement 15 en-têtes de sécurité
- Installation et utilisation :
- Avant Helmet : réponse HTTP avec en-têtes basiques (Content-Type seulement)
- Après Helmet : 12+ en-têtes de sécurité ajoutés automatiquement

Les en-têtes de sécurité expliqués

Chaque en-tête protège contre une menace spécifique

En-tête	Protection	Menace bloquée
Content-Security-Policy	Contrôle les sources de scripts/styles	XSS, injection de code
Strict-Transport-Security	Force HTTPS	Attaques man-in-the-middle
X-Frame-Options	Empêche l'intégration en iframe	Clickjacking
X-Content-Type-Options	Empêche le MIME sniffing	Exécution de fichiers malveillants
X-XSS-Protection	Active le filtre XSS du navigateur	XSS réfléchi
Referrer-Policy	Contrôle les informations de référent	Fuite d'informations

Rate Limiting au niveau Express

Protéger vos routes sensibles avec express-rate-limit

```
const rateLimit = require('express-rate-limit');

// Limite générale : 100 requêtes par 15 minutes
const apiLimiter = rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 100,
    message: 'Trop de requêtes, réessayez plus tard.'
});
app.use('/api/', apiLimiter);

// Limite stricte pour le login : 5 tentatives par heure
const loginLimiter = rateLimit({
    windowMs: 60 * 60 * 1000,
    max: 5,
    message: 'Trop de tentatives de connexion.'
});
app.use('/api/login', loginLimiter);
```

CORS et HPP : deux middlewares essentiels

Contrôler les origines et protéger les paramètres

```
const cors = require('cors');
app.use(cors({
    origin: 'https://monsite.fr', // Seul ce domaine est autorisé
    methods: ['GET', 'POST'],
    credentials: true
}));
```

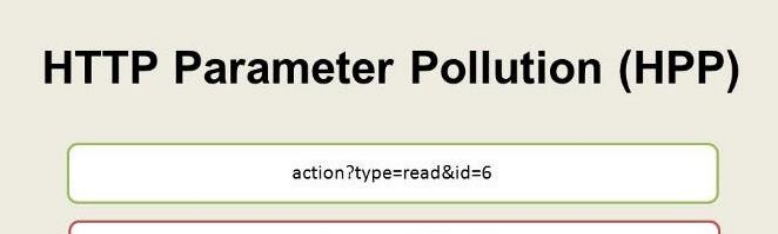
```
const hpp = require('hpp');
app.use(hpp()); // Empêche la duplication malveillante de paramètres
```

- **CORS** (Cross-Origin Resource Sharing) : contrôle quels domaines peuvent accéder à votre API
- **HPP** (HTTP Parameter Pollution) : protège contre la manipulation des paramètres

Le flux complet d'une requête sécurisée

Chaque middleware ajoute une couche de protection

- Diagramme séquentiel du flux d'une requête à travers les middlewares Express :
- **1.** Requête HTTP entrante
- **2. express-rate-limit** → Vérifie le nombre de requêtes (429 si dépassé)
- **3. cors** → Vérifie l'origine de la requête (403 si non autorisé)
- **4. hpp** → Nettoie les paramètres dupliqués
- **5. helmet** → Ajoute les en-têtes de sécurité à la réponse
- **6. Route Handler** → Traite la logique métier
- **7.** Réponse HTTP sécurisée envoyée au client



Qu'est-ce que l'OWASP ?

L'organisation de référence pour la sécurité des applications web

- **OWASP** = Open Web Application Security Project (fondé en 2001)
- Organisation à but non lucratif dédiée à la sécurité logicielle
- Publie le **Top 10** : les 10 risques de sécurité les plus critiques pour les applications web
- Dernière version : OWASP Top 10 **2021** (mise à jour régulière)
- Utilisé comme référence par les entreprises, les gouvernements et les universités du monde entier

- Le Top 10 n'est pas une liste exhaustive, mais un point de départ essentiel

Qu'est-ce que l'OWASP ?

L'organisation de référence pour la sécurité des applications web

owasp.org/Top10/2025/

Home | Search | OWASP/Top10 ☆ 5.3k ⑂ 1k

OWASP Top 10:2025
- Home
- Introduction
- About OWASP
- What are Application Security Risks?
- Establishing a Modern Application Security Program

Top 10:2025 List
- A01 Broken Access Control
- A02 Security Misconfiguration
- A03 Software Supply Chain Failures
- A04 Cryptographic Failures
- A05 Injection
- A06 Insecure Design
- A07 Authentication Failures
- A08 Software or Data Integrity Failures
- A09 Security Logging and Alerting Failures
- A10 Mishandling of Exceptional Conditions

Next Steps

Start with the Introduction to learn about what's new in the 2025 version.

Navigation

- Introduction
- About OWASP
- What are Application Security Risks?
- Establishing a Modern Application Security Program

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

Note: Translations will be added as they become available.

Table of contents
- About This Release
- Main Project Page
- Getting Started
- Navigation
  - Top 10:2025 List

Vue d'ensemble de l'OWASP Top 10

Les 10 vulnérabilités les plus critiques

#	Catégorie	Description courte
A01	Contrôle d'accès défaillant	Accès non autorisé à des ressources
A02	Défaillances cryptographiques	Données sensibles mal protégées
A03	Injection	Code malveillant injecté dans les requêtes
A04	Conception non sécurisée	Absence de contrôles de sécurité dès la conception
A05	Mauvaise configuration	Paramètres de sécurité incorrects
A06	Composants vulnérables	Bibliothèques obsolètes avec des failles connues
A07	Authentification défaillante	Gestion incorrecte de l'identité

A01 : Contrôle d'accès défaillant (Broken Access Control)

La vulnérabilité n°1 la plus fréquente

- **Définition** : Un utilisateur peut accéder à des ressources ou effectuer des actions au-delà de ses permissions
- Deux types principaux :

- **Escalade verticale** : un utilisateur normal accède aux fonctions admin

- **Escalade horizontale** : un utilisateur accède aux données d'un autre utilisateur

VULNÉRABLE

```
// DANGEREUX : l'ID utilisateur vient de l'URL !
app.get('/api/user/:id', (req, res) => {
    const userId = req.params.id; // L'attaquant peut changer l'ID
    db.get('SELECT * FROM users WHERE id = ?', [userId], (err, user) => {
        res.json(user); // Données d'un autre utilisateur exposées !
    });
});
```

A02 : Défaillances cryptographiques

Protéger les données sensibles avec un chiffrement fort

VULNÉRABLE

Code VULNÉRABLE — Hachage MD5 :

```
const crypto = require('crypto');
// DANGEREUX : MD5 est cassé, pas de sel !
const hash = crypto.createHash('md5').update(password).digest('hex');
```

SÉCURISÉ

Code SÉCURISÉ — bcrypt avec sel :

```
const bcrypt = require('bcrypt');
const saltRounds = 12;
// SÉCURISÉ : bcrypt est lent et utilise un sel unique
const hash = await bcrypt.hash(password, saltRounds);
// Vérification
const match = await bcrypt.compare(password, hash);
```

- **Définition** : Utilisation de méthodes de chiffrement faibles ou absence de chiffrement
- Exemples de données sensibles : mots de passe, numéros de carte bancaire, données personnelles

A03 : Injection SQL

Comment un attaquant peut voler toute votre base de données

- **Définition** : L'attaquant injecte du code SQL malveillant via les entrées utilisateur
- Diagramme étape par étape de l'attaque :

1. L'attaquant entre `' OR '1'='1` dans le champ username
2. Le serveur construit la requête : `SELECT * FROM users WHERE username = '' OR '1'='1`
3. La condition `'1'='1` est toujours vraie → toutes les lignes sont retournées
4. L'attaquant obtient l'accès sans connaître le mot de passe

VULNÉRABLE

```
// DANGEREUX : concaténation directe !
const query = `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`;
db.get(query, (err, row) => { /* ... */ });
```

SÉCURISÉ

A03 : Injection

Visualiser la différence entre code vulnérable et sécurisé

- Diagramme comparatif côte à côte :
- **Flux vulnérable** : Entrée utilisateur → Concaténation dans la requête SQL → Base de données exécute le code malveillant → Données volées
- **Flux sécurisé** : Entrée utilisateur → Requête paramétrée (données séparées du code) → Base de données traite l'entrée comme une simple valeur → Accès refusé si invalide
- Le principe fondamental : **ne jamais faire confiance aux entrées utilisateur**

A04 : Conception non sécurisée (Insecure Design)

La sécurité doit être pensée dès la conception

- **Définition** : Absence de contrôles de sécurité dans l'architecture de l'application
- Ce n'est pas un bug de code, c'est un **défaut de conception**
- Exemples :

- Pas de rate limiting sur les endpoints sensibles

- Pas de validation des règles métier (ex: transfert d'argent négatif)

- Pas de modélisation des menaces (threat modeling)

```
// Exemple : validation des règles métier
app.post('/transfer', authenticate, (req, res) => {
    const { amount, toAccount } = req.body;
    if (amount <= 0) return res.status(400).json({ error: 'Montant invalide' });
    if (amount > req.user.balance) return res.status(400).json({ error: 'Solde insuffisant' });
```

A05 : Mauvaise configuration de sécurité

Les erreurs de configuration sont les plus faciles à exploiter

- **Définition** : Paramètres de sécurité incorrects, par défaut ou manquants
- Exemples fréquents en Node.js :

- Mode debug activé en production → stack traces exposées

- Identifiants par défaut non changés

- En-têtes de sécurité manquants

- CORS trop permissif (`origin: '*'`)

VULNÉRABLE

Code VULNÉRABLE :

```
// DANGEREUX : affiche les détails de l'erreur !
app.use((err, req, res, next) => {
    res.status(500).json({
        error: err.message,
        stack: err.stack // L'attaquant voit la pile d'appels !
    });
});
```

A06 : Composants vulnérables et obsolètes

Vos dépendances npm peuvent être votre plus grande faiblesse

```
# Vérifier les vulnérabilités de vos dépendances
npm audit

# Corriger automatiquement les vulnérabilités
npm audit fix

# Voir les packages obsolètes
npm outdated
```

- **Définition** : Utilisation de bibliothèques avec des vulnérabilités connues
- npm compte plus de 2 millions de packages — beaucoup ont des failles
- Commandes essentielles :

A07 : Authentification défaillante et JWT

Les erreurs courantes avec les JSON Web Tokens

- **Définition** : Mauvaise implémentation de l'authentification et de la gestion des sessions
- Erreurs fréquentes avec JWT :

- Secret trop faible ou codé en dur

- Données sensibles dans le payload (mot de passe !)

- Tokens sans expiration

- Pas de rotation des tokens

VULNÉRABLE

Code VULNÉRABLE :

```
// DANGEREUX : mot de passe dans le payload !
const token = jwt.sign({
    userId: user.id,
    email: user.email,
    password: user.password, // JAMAIS faire ça !
    role: user.role
}, 'secret123'); // Secret trop faible !
```

A08 à A10 : Intégrité, Journalisation et SSRF

Trois vulnérabilités souvent négligées

- **A08 — Intégrité logicielle** : Vérifier l'intégrité des dépendances et des mises à jour

- Utiliser `npm ci` au lieu de `npm install` en production

- Vérifier les hashes SRI pour les scripts CDN

```
const winston = require('winston');
const logger = winston.createLogger({
    transports: [new winston.transports.File({ filename: 'security.log' })]
});
// Logger chaque tentative de connexion
logger.info({ event: 'LOGIN_ATTEMPT', ip: req.ip, email: req.body.email });
```

- **A09 — Journalisation insuffisante** : Sans logs, impossible de détecter une attaque

XSS : Cross-Site Scripting expliqué

L'attaque qui transforme votre site en arme

- **Définition** : L'attaquant injecte du JavaScript malveillant qui s'exécute dans le navigateur des victimes

- 3 types de XSS :

- **XSS Réfléchi** : le script est dans l'URL, exécuté quand la victime clique

- **XSS Stocké** : le script est sauvegardé en base, exécuté pour tous les visiteurs

- **XSS DOM** : le script manipule le DOM côté client

VULNÉRABLE

```
// VULNÉRABLE : le commentaire est affiché sans échappement
app.get('/comments', (req, res) => {
    const comments = db.all('SELECT * FROM comments');
    // Si un commentaire contient <script>alert('XSS')</script>
    // Le script s'exécute dans le navigateur de chaque visiteur !
    res.send(comments.map(c => `<p>${c.text}</p>`).join(''));
});
```

Défenses contre le XSS

Échapper, valider, et contrôler

```
const escapeHtml = require('escape-html');
const safeComment = escapeHtml(userComment);
// <script> devient <script> → non exécutable
```

```
app.use(helmet.contentSecurityPolicy({
    directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'"], // Seuls les scripts du même domaine
        styleSrc: ["'self'", "'unsafe-inline'"]
    }
}));
```

- **1. Échappement des sorties** :
- **2. Content Security Policy (CSP)** via Helmet :

Récapitulatif : les bonnes pratiques de sécurité Node.js

Votre checklist de sécurité

Catégorie	Action	Outil/Méthode
Infrastructure	Utiliser un reverse proxy	NGINX
En-têtes HTTP	Configurer les headers de sécurité	Helmet.js
Rate Limiting	Limiter les requêtes	express-rate-limit + NGINX
Injection	Requêtes paramétrées	Prepared statements
XSS	Échapper les sorties	escape-html, CSP
Authentification	JWT sécurisé avec expiration	jsonwebtoken + bcrypt

Ressources et références

Pour aller plus loin

- **OWASP Top 10** : https://owasp.org/Top10/
- **OWASP Node.js Security Cheat Sheet** : https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html
- **Helmet.js** : https://helmetjs.github.io/
- **Express Security Best Practices** : https://expressjs.com/en/advanced/best-practice-security.html

- **OWASP NodeGoat** : https://github.com/OWASP/NodeGoat
- **Snyk** : https://snyk.io/
- Contact : **joseph.azar@umlp.fr**

TP : Capture The Flag

Mettez vos compétences à l'épreuve !

- Vous allez recevoir l'accès à une **application web volontairement vulnérable**
- Votre mission : trouver et exploiter le **maximum de vulnérabilités**
- L'application ressemble à une vraie application — aucun indice n'est donné !
- Vous devez penser comme un **attaquant** (ethical hacker)
- Explorez tout : formulaires, URLs, cookies, tokens, requêtes réseau…